

1. WAP to create procedure to display data of employee;

```
CREATE TABLE employees (  
  employee_id NUMBER PRIMARY KEY,  
  first_name VARCHAR2(50),  
  last_name VARCHAR2(50),  
  email VARCHAR2(100),  
  job_id VARCHAR2(20),  
  salary NUMBER(10, 2)  
);
```

```
-- Sample insert to test the procedure
```

```
INSERT INTO employees VALUES (1, 'John', 'Doe', 'john.doe@example.com', 'DEV01', 5000);  
INSERT INTO employees VALUES (2, 'Jane', 'Smith', 'jane.smith@example.com', 'HR01',  
4500);  
COMMIT;
```

```
-- To enable output display in SQL*Plus  
SET SERVEROUTPUT ON;
```

```
CREATE OR REPLACE PROCEDURE display_employee IS  
BEGIN
```

```
-- Loop through each employee record and display details
```

```
FOR emp_rec IN (SELECT * FROM employees) LOOP
```

```
  DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_rec.employee_id);
```

```
  DBMS_OUTPUT.PUT_LINE('Name: ' || emp_rec.first_name || ' ' || emp_rec.last_name);
```

```
  DBMS_OUTPUT.PUT_LINE('Email: ' || emp_rec.email);
```

```
  DBMS_OUTPUT.PUT_LINE('Job ID: ' || emp_rec.job_id);
```

```
  DBMS_OUTPUT.PUT_LINE('Salary: ' || emp_rec.salary);
```

```
  DBMS_OUTPUT.PUT_LINE('-----');
```

```
END LOOP;
```

```
END;
```

```
/
```

```
-- Then call the procedure:
```

```
EXEC display_employee;
```

2. WAP to create procedure to display data of employee given by user;

```
CREATE TABLE employees (  
    employee_id NUMBER PRIMARY KEY,  
    first_name VARCHAR2(50),  
    last_name VARCHAR2(50),  
    email VARCHAR2(100),  
    job_id VARCHAR2(20),  
    salary NUMBER(10, 2)  
);  
  
-- Sample insert to test the procedure  
INSERT INTO employees VALUES (1, 'John', 'Doe', 'john.doe@example.com', 'DEV01', 5000);  
INSERT INTO employees VALUES (2, 'Jane', 'Smith', 'jane.smith@example.com', 'HR01',  
4500);  
COMMIT;  
  
-- To enable output display in SQL*Plus  
SET SERVEROUTPUT ON;  
  
CREATE OR REPLACE PROCEDURE display_employee_by_id(p_employee_id IN NUMBER) IS  
    emp_rec employees%ROWTYPE;  
BEGIN  
    SELECT * INTO emp_rec FROM employees WHERE employee_id = p_employee_id;  
  
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_rec.employee_id);  
    DBMS_OUTPUT.PUT_LINE('Name: ' || emp_rec.first_name || ' ' || emp_rec.last_name);  
    DBMS_OUTPUT.PUT_LINE('Email: ' || emp_rec.email);  
    DBMS_OUTPUT.PUT_LINE('Job ID: ' || emp_rec.job_id);  
    DBMS_OUTPUT.PUT_LINE('Salary: ' || emp_rec.salary);  
    DBMS_OUTPUT.PUT_LINE('-----');  
  
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        DBMS_OUTPUT.PUT_LINE('No employee found with ID: ' || p_employee_id);  
END;  
/  
  
-- Then call the procedure:  
EXEC display_employee_by_id(2);
```

3. WAP to create procedure to update salary 5% of “Manager”;

```
CREATE TABLE employees (  
    employee_id NUMBER PRIMARY KEY,  
    first_name VARCHAR2(50),  
    last_name VARCHAR2(50),  
    email VARCHAR2(100),  
    job_id VARCHAR2(20),  
    salary NUMBER(10, 2)  
);  
  
-- Sample insert to test the procedure  
INSERT INTO employees VALUES (1, 'John', 'Doe', 'john.doe@example.com', 'DEV01', 5000);  
INSERT INTO employees VALUES (2, 'Jane', 'Smith', 'jane.smith@example.com', 'HR01',  
4500);  
INSERT INTO employees VALUES (3, 'Alice', 'Manager', 'alice.manager@example.com',  
'MANAGER01', 6000);  
COMMIT;  
  
-- To enable output display in SQL*Plus  
SET SERVEROUTPUT ON;  
  
CREATE OR REPLACE PROCEDURE update_manager_salary IS  
BEGIN  
    UPDATE employees  
    SET salary = salary * 1.05  
    WHERE LOWER(job_id) LIKE '%manager%';  
  
    COMMIT;  
    DBMS_OUTPUT.PUT_LINE('Manager salaries updated by 5%. Rows affected: ' ||  
SQL%ROWCOUNT);  
END;  
/  
  
-- Then call the procedure:  
EXEC update_manager_salary;
```

4. WAP To create procedure to calculate 10% bonus of given emp no;

```
CREATE TABLE employees (  
  employee_id NUMBER PRIMARY KEY,  
  first_name VARCHAR2(50),  
  last_name VARCHAR2(50),  
  email VARCHAR2(100),  
  job_id VARCHAR2(20),  
  salary NUMBER(10, 2)  
);
```

```
INSERT INTO employees VALUES (1, 'John', 'Doe', 'john.doe@example.com', 'DEV01', 5000);  
INSERT INTO employees VALUES (2, 'Jane', 'Smith', 'jane.smith@example.com', 'HR01',  
4500);  
INSERT INTO employees VALUES (3, 'Alice', 'Manager', 'alice.manager@example.com',  
'MANAGER01', 6000);  
COMMIT;
```

```
-- Enable server output in your environment before executing this file  
SET SERVEROUTPUT ON;
```

```
CREATE OR REPLACE PROCEDURE calculate_bonus(p_employee_id IN NUMBER) IS  
  v_salary employees.salary%TYPE;  
  v_bonus NUMBER(10, 2);  
BEGIN  
  SELECT salary INTO v_salary FROM employees WHERE employee_id = p_employee_id;  
  
  v_bonus := v_salary * 0.10;  
  
  DBMS_OUTPUT.PUT_LINE('Employee ID: ' || p_employee_id);  
  DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);  
  DBMS_OUTPUT.PUT_LINE('10% Bonus: ' || v_bonus);  
  
EXCEPTION  
  WHEN NO_DATA_FOUND THEN  
    DBMS_OUTPUT.PUT_LINE('No employee found with ID: ' || p_employee_id);  
END;  
/
```

```
-- Replace the error line with a valid test call:  
EXEC calculate_bonus(1);
```

5. WAP to create procedure to delete data of an employee whose designation is "Manager";

```
CREATE TABLE employees (  
  employee_id NUMBER PRIMARY KEY,  
  first_name VARCHAR2(50),  
  last_name VARCHAR2(50),  
  email VARCHAR2(100),  
  job_id VARCHAR2(20),  
  salary NUMBER(10, 2)  
);
```

```
INSERT INTO employees VALUES (1, 'John', 'Doe', 'john.doe@example.com', 'DEV01', 5000);  
INSERT INTO employees VALUES (2, 'Jane', 'Smith', 'jane.smith@example.com', 'HR01',  
4500);  
INSERT INTO employees VALUES (3, 'Alice', 'Manager', 'alice.manager@example.com',  
'MANAGER01', 6000);  
COMMIT;
```

```
-- Enable server output in your environment before executing this file  
SET SERVEROUTPUT ON;
```

```
CREATE OR REPLACE PROCEDURE delete_manager_employees IS  
BEGIN  
  DELETE FROM employees WHERE LOWER(job_id) LIKE '%manager%';  
  
  COMMIT;  
  DBMS_OUTPUT.PUT_LINE('Deleted employees with designation containing "Manager".  
Rows affected: ' || SQL%ROWCOUNT);  
EXCEPTION  
  WHEN OTHERS THEN  
    DBMS_OUTPUT.PUT_LINE('Error occurred: ' || SQLERRM);  
END;  
/
```

```
-- Replace the error line with a valid test call:  
EXEC delete_manager_employees;
```